Task 3 — Secure Code Assessment Report

User Management CLI | CodSoft Cyber Security Internship

CodSoft Cyber Security Internship

02 November 2026

Contents

1. Executive summary . . . 1
2. Scope and methodology . . . 1
3. Findings in the vulnerable baseline . . . 2
   SCA-01 — SQL injection . . . 2
   SCA-02 — Plaintext password storage . . . 2
   SCA-03 — Missing input validation . . . 2
   SCA-04 — Production authentication controls not represented . . . 3
4. Remediation summary . . . 3
5. Automated verification . . . 3
   Local Windows verification . . . 3
6. Residual risks and limitations . . . 3
7. Conclusion . . . 3
1. Parameterized SQL . . . 4
2. Hash passwords . . . 4
3. Validate input . . . 4
4. Use generic authentication feedback . . . 4
5. Add before production deployment . . . 4

\newpage # Secure Code Assessment Report

Project: User Management CLI
Language: Python
Database: SQLite
Assessment: Manual source review + focused automated tests + static analysis
Organization: CodSoft — Cyber Security Internship
Task: 3 — Secure Code Assessment

1. Executive summary

This assessment reviews a small local user-registration and authentication application. The original training sample contains deliberately insecure code so that common weaknesses can be identified and explained. A separate secure implementation demonstrates the recommended fixes.

The most important issues in the vulnerable sample are SQL injection through dynamically constructed SQL statements and storage of passwords in plaintext. The remediated implementation uses parameterized SQL statements, Werkzeug's password-hashing utilities, input validation, generic authentication feedback, and automated tests.

Assessment boundary: This is a focused educational code review, not a production certification, penetration test, or proof that every possible weakness has been found.

2. Scope and methodology

Files reviewed: - `app/vulnerable_app.py` — intentionally insecure baseline. - `app/secure_app.py` — remediated implementation. - `tests/test_secure_app.py` — regression and security-focused tests.

Review activities: 1. Manually traced registration and authentication data flow. 2. Checked SQL query construction and parameter binding. 3. Checked how passwords are stored and verified. 4. Reviewed input

validation and authentication responses. 5. Ran pytest, Bandit, and Ruff against the secure implementation (record actual results after execution).

The vulnerable sample is intentionally excluded from static-scan pass criteria because the SQL construction is deliberately unsafe for teaching purposes.

3. Findings in the vulnerable baseline

ID	Severity	Finding	Impact
SCA-01	High	SQL injection from string-built queries	User-controlled values can alter the meaning of database queries.
SCA-02	High	Plaintext password storage	Anyone who obtains the database can read users' passwords directly.
SCA-03	Medium	Missing input validation	Unbounded or unexpected usernames/passwords can be accepted, causing inconsistent data and avoidable resource use.
SCA-04	Low	No controlled authentication policy in the sample	A production-facing authentication service would need attempt throttling, monitoring, and account-abuse controls.

SCA-01 — SQL injection

Evidence: `vulnerable_app.py` builds INSERT and SELECT statements using f-strings that contain `username` and `password`.

Risk: Untrusted values are interpreted as SQL syntax rather than strictly as data. This can bypass intended query conditions or alter database operations.

Remediation: Use parameterized statements with placeholders. The secure implementation passes user values separately as query parameters.

Validation: A regression test confirms that an SQL-like username is treated as ordinary data and does not authenticate as another user.

SCA-02 — Plaintext password storage

Evidence: The baseline schema stores a column named `password` and inserts the raw password value.

Risk: A database disclosure exposes actual passwords, which users may have reused on other services.

Remediation: Store a salted, computationally expensive password hash using a maintained password-hashing library. Verify passwords with the library's verification function. Never log or display passwords.

Validation: A test reads the saved database value and confirms it differs from the supplied password and uses the expected scrypt hash format.

SCA-03 — Missing input validation

Evidence: The baseline accepts usernames and passwords without explicit length or character rules.

Risk: Unexpected and excessively long values can enter the database and make account handling inconsistent.

Remediation: Enforce a documented username format and password length limits before database operations.

Validation: Tests reject malformed usernames and passwords shorter than the configured minimum.

SCA-04 — Production authentication controls not represented

Evidence: The baseline is a minimal local demonstration and has no attempt throttling or security-event monitoring.

Risk: If a future network-facing login service copied this design without additional controls, repeated password guesses could go undetected or unthrottled.

Remediation: Before any network deployment, add rate limiting, monitoring, secure recovery, MFA where appropriate, and an operational incident-response process. These controls are outside this local CLI prototype's scope.

4. Remediation summary

Control	Vulnerable baseline	Secure implementation
SQL queries	String interpolation	Parameterized SQL
Password storage	Plaintext	Salted scrypt hash via Werkzeug
Input validation	Not defined	Username allowlist and password length bounds
Authentication response	Direct query result	Boolean success/failure; generic failure message
Regression tests	Not provided	Focused pytest suite

5. Automated verification

Verified GitHub Actions run on 2 November 2026: Task 3 Secure Code Checks.

Check	Command	Verified result
Unit/security tests	`python -m pytest "Task - 3/tests" -v`	PASS — 7 tests passed
Bandit	`python -m bandit -q "Task - 3/app/secure_app.py"`	PASS — no findings reported
Ruff	`python -m ruff check "Task - 3/app/secure_app.py" "Task - 3/tests"`	PASS — no lint errors

These results apply to the secure implementation and listed test suite in the cited CI run. Keep screenshots of the run or terminal output in the `screenshots/` folder if separate submission evidence is required.

Local Windows verification

On 2 November 2026, the project was also run locally in the user's Windows PowerShell environment using the project virtual environment. The displayed pytest output confirmed **7 passed in 0.95s**. Registration and successful/unsuccessful login were also demonstrated locally. A separate screenshot should be retained in `screenshots/` as submission evidence.

6. Residual risks and limitations

- This is a local command-line demonstration, not a web application.
- No rate limiter, MFA, password reset, audit logging, or account lockout is implemented.
- SQLite is suitable for this small local learning project; production deployment needs a broader architecture and operational review.
- Dependencies should be kept updated and checked for known vulnerabilities before distribution.
- The vulnerable training sample must never be deployed or used with real accounts.

7. Conclusion

The review identified two high-severity weaknesses in the intentionally vulnerable baseline: SQL injection and plaintext password storage. The secure implementation demonstrates parameterized SQL, password hashing,

validation, and regression tests. The project provides a clear before-and-after example of secure coding practices for the CodSoft Task 3 requirement.

\newpage

1. Parameterized SQL

Avoid:

```
query = f"SELECT * FROM users WHERE username = '{username}'"
connection.execute(query)
```

Use:

```
connection.execute(
    "SELECT password_hash FROM users WHERE username = ?",
    (username,),
)
```

A placeholder keeps user input separate from SQL syntax.

2. Hash passwords

Avoid: Saving the user's original password in the database.

Use: A maintained password-hashing function such as Werkzeug's `generate_password_hash` and `check_password_hash`. Store only the resulting hash; never store or log the original password.

3. Validate input

Apply server-side validation before database operations. Define allowed username characters and sensible length limits. Enforce password length requirements and avoid silently truncating passwords.

4. Use generic authentication feedback

Return a single failure message such as "Invalid username or password." Do not reveal whether an account exists.

5. Add before production deployment

A network-facing service should additionally use: - Rate limiting and monitoring for repeated failed logins. - Multi-factor authentication where appropriate. - Secure password recovery and session handling. - HTTPS, security headers, backups, and access controls. - Dependency updates and vulnerability monitoring. - Security logging that never records passwords or authentication tokens.

The Task 3 secure app is a local demonstration and does not claim to implement all production controls.